```
# ============================================
# 1. CONFIGURACIÓN DEL ENTORNO (GOOGLE COLAB)
# ============================================

!pip install -q streamlit pyngrok pandas numpy scikit-learn plotly
!pip install -q streamlit-plotly-events
# Permite ejecutar Streamlit dentro de Colab
from pyngrok import ngrok
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 56.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 90.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.8/14.8 MB 90.1 MB/s eta 0:00:00
```

```
# ============================================
# 2. IMPORTACIÓN DE LIBRERÍAS
# ============================================

import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

```
# ============================================
# 3. CARGA DINÁMICA DEL DATASET
# ============================================

def load_dataset(uploaded_file):
    """
    Carga el dataset de forma dinámica.
    Permite reutilizar el sistema con diferentes fuentes de datos.
    """
    if uploaded_file.name.endswith('.csv'):
        return pd.read_csv(uploaded_file)
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
        st.error("Formato no soportado")
        return None
```

```
# ============================================
# 4. CÁLCULO DE VARIABLES RFM
# ============================================

def calculate_rfm(df, customer_col, date_col, amount_col):
    """
    Implementa el análisis RFM descrito en la propuesta de titulación.
    """

    df[date_col] = pd.to_datetime(df[date_col])

    reference_date = df[date_col].max() + pd.Timedelta(days=1)

    rfm = df.groupby(customer_col).agg({
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })

    rfm.columns = ['Recency', 'Frequency', 'Monetary']

    return rfm.reset_index()
```

```
# ============================================
# 5. NORMALIZACIÓN Y MÉTRICAS DE VALIDACIÓN
# ============================================

def scale_features(rfm_df):
    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_df[['Recency','Frequency','Monetary']])
    return scaled
def calculate_elbow_silhouette(data, max_k=10):
    inertia = []
    silhouette = []

    for k in range(2, max_k+1):
```

```
        model = KMeans(n_clusters=k, random_state=42)
        labels = model.fit_predict(data)
        inertia.append(model.inertia_)
        silhouette.append(silhouette_score(data, labels))

    return inertia, silhouette
```

```
# ==========================================
# 6. ENTRENAMIENTO DEL MODELO K-MEANS (K = 4)
# ==========================================

def train_kmeans_fixed(data):
    """
    Entrena el modelo K-Means con K=4 clusters,
    alineado a los segmentos definidos en el proyecto:
    VIP, Leales, En Riesgo, Perdidos.
    """
    kmeans = KMeans(n_clusters=4, random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels
```

```
# ==========================================
# 7. ASIGNACIÓN DE SEGMENTOS RFM (AJUSTADA)
# ==========================================

def assign_rfm_segments(rfm):
    """
    Segmentación alineada a literatura RFM y al documento del proyecto.
    Perdidos: alta recencia + bajo valor monetario.
    """

    r25, r75 = rfm['Recency'].quantile([0.25, 0.75])
    f25, f75 = rfm['Frequency'].quantile([0.25, 0.75])
    m25, m75 = rfm['Monetary'].quantile([0.25, 0.75])

    def segment(row):
        # VIP
        if row['Recency'] <= r25 and row['Frequency'] >= f75 and row['Monetary'] >= m75:
            return 'VIP'

        # Leales
        elif row['Recency'] <= r75 and row['Frequency'] >= f25 and row['Monetary'] >= m25:
            return 'Leales'

        # En Riesgo
        elif row['Recency'] > r75 and row['Frequency'] >= f25 and row['Monetary'] >= m25:
            return 'En Riesgo'

        # Perdidos (AJUSTE CLAVE)
        elif row['Recency'] > r75 and row['Monetary'] <= m25:
            return 'Perdidos'

        else:
            return 'Leales'

    rfm['Segmento'] = rfm.apply(segment, axis=1)
    return rfm
```

```
# ==========================================
# 8. RECOMENDACIONES ESTRATÉGICAS DE MARKETING
# ==========================================

def marketing_recommendations(segment):
    recommendations = {
        "VIP": [
            "Ofrecer programas de fidelización exclusivos",
            "Acceso anticipado a promociones y lanzamientos",
            "Comunicación personalizada 1 a 1",
            "Incentivos por referidos"
        ],
        "Leales": [
            "Descuentos progresivos por frecuencia de compra",
            "Campañas de cross-selling y up-selling",
            "Programas de puntos",
            "Encuestas de satisfacción"
        ],
        "En Riesgo": [
            "Campañas de reactivación con descuentos agresivos",
            "Recordatorios personalizados por email/SMS",
            "Ofertas por tiempo limitado",
```

```
            "Análisis de causas de abandono"
        ],
        "Perdidos": [
            "Campañas de recuperación de bajo costo",
            "Ofertas de bienvenida para reenganche",
            "Excluir de campañas costosas",
            "Evaluar eliminación del CRM activo"
        ]
    }

    return recommendations.get(segment, [])
```

```
# ==========================================
# 9. DASHBOARD INTERACTIVO STREAMLIT
# ==========================================

st.title("Segmentación de Clientes – Retail (RFM + K-Means)")

uploaded_file = st.file_uploader("Cargar dataset transaccional")

if uploaded_file:
    df = load_dataset(uploaded_file)

    customer_col = st.selectbox("Columna ID Cliente", df.columns)
    date_col = st.selectbox("Columna Fecha", df.columns)
    amount_col = st.selectbox("Columna Monto", df.columns)

    rfm = calculate_rfm(df, customer_col, date_col, amount_col)
    scaled = scale_features(rfm)

    model, labels = train_kmeans_fixed(scaled)
    rfm['Cluster'] = labels

    rfm = assign_rfm_segments(rfm)

    st.subheader("Distribución de Segmentos")
    st.bar_chart(rfm['Segmento'].value_counts())

    fig = px.scatter_3d(
        rfm,
        x='Recency',
        y='Frequency',
        z='Monetary',
        color='Segmento',
        title="Visualización 3D de Segmentación de Clientes"
    )
    st.plotly_chart(fig)

    # ----------------------------
    # RECOMENDACIONES
    # ----------------------------
    st.subheader("Recomendaciones de Marketing")

    selected_segment = st.selectbox(
        "Selecciona un segmento",
        ['VIP', 'Leales', 'En Riesgo', 'Perdidos']
    )

    recs = marketing_recommendations(selected_segment)

    for r in recs:
        st.markdown(f"✅ {r}")
```

```
2026-02-19 15:07:44.336 WARNING streamlit.runtime.scriptrunner_utils.script_run_context: Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.649
  Warning: to view this Streamlit app on a browser, run it with the following
  command:

    streamlit run /usr/local/lib/python3.12/dist-packages/colab_kernel_launcher.py [ARGUMENTS]
2026-02-19 15:07:44.651 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.652 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.655 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.656 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.658 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.660 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.662 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
2026-02-19 15:07:44.662 Thread 'MainThread': missing ScriptRunContext! This warning can be ignored when running in bare mode.
```

```
# Ejecutar Streamlit en Colab
# Obtén tu authtoken de ngrok en https://dashboard.ngrok.com/get-started/your-authtoken
# y reemplaza 'YOUR_AUTHTOKEN' con tu token real.
ngrok.set_auth_token('38CulDX1M53o6OGkHScEziK918U_4TJ3Erf2M63j3FoxhJqPX')
public_url = ngrok.connect(8501)
print(public_url)
```

```
print(public_url)

# Guardar el código de Streamlit en un archivo app.py
with open('app.py', 'w') as f:
    f.write("""
import pandas as pd
import numpy as np
import streamlit as st
import plotly.express as px

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from streamlit_plotly_events import plotly_events

def load_dataset(uploaded_file):
    if uploaded_file.name.endswith('.csv'):
        return pd.read_csv(uploaded_file)
    elif uploaded_file.name.endswith('.xlsx'):
        return pd.read_excel(uploaded_file)
    else:
        st.error("Formato no soportado")
        return None

def calculate_rfm(df, customer_col, date_col, amount_col):
    df[date_col] = pd.to_datetime(df[date_col])
    reference_date = df[date_col].max() + pd.Timedelta(days=1)
    rfm = df.groupby(customer_col).agg({
        date_col: lambda x: (reference_date - x.max()).days,
        customer_col: 'count',
        amount_col: 'sum'
    })
    rfm.columns = ['Recency', 'Frequency', 'Monetary']
    return rfm.reset_index()

def scale_features(rfm_df):
    scaler = StandardScaler()
    scaled = scaler.fit_transform(rfm_df[['Recency','Frequency','Monetary']])
    return scaled

def calculate_elbow_silhouette(data, max_k=10):
    inertia = []
    silhouette = []
    for k in range(2, max_k+1):
        model = KMeans(n_clusters=k, random_state=42)
        labels = model.fit_predict(data)
        inertia.append(model.inertia_)
        silhouette.append(silhouette_score(data, labels))
    return inertia, silhouette

def train_kmeans_fixed(data):
    kmeans = KMeans(n_clusters=4, random_state=42)
    labels = kmeans.fit_predict(data)
    return kmeans, labels

def assign_rfm_segments(rfm):
    r25, r75 = rfm['Recency'].quantile([0.25, 0.75])
    f25, f75 = rfm['Frequency'].quantile([0.25, 0.75])
    m25, m75 = rfm['Monetary'].quantile([0.25, 0.75])

    def segment(row):
        # VIP
        if row['Recency'] <= r25 and row['Frequency'] >= f75 and row['Monetary'] >= m75:
            return 'VIP'

        # Leales
        elif row['Recency'] <= r75 and row['Frequency'] >= f25 and row['Monetary'] >= m25:
            return 'Leales'

        # En Riesgo
        elif row['Recency'] > r75 and row['Frequency'] >= f25 and row['Monetary'] >= m25:
            return 'En Riesgo'

        # Perdidos (AJUSTE CLAVE)
        elif row['Recency'] > r75 and row['Monetary'] <= m25:
            return 'Perdidos'

        else:
            return 'Leales'

    rfm['Segmento'] = rfm.apply(segment, axis=1)
    return rfm

def marketing_recommendations(segment):
    return {
        "VIP": [
            "Programas de fidelización premium",
            "Ofertas exclusivas y lanzamientos anticipados",
```

```
                "Beneficios por referidos",
                "Atención personalizada"
            ],
            "Leales": [
                "Promociones por frecuencia",
                "Cross-selling y up-selling",
                "Programa de puntos",
                "Comunicación periódica personalizada"
            ],
            "En Riesgo": [
                "Campañas de reactivación",
                "Descuentos por tiempo limitado",
                "Recordatorios personalizados",
                "Análisis de churn"
            ],
            "Perdidos": [
                "Campañas de recuperación de bajo costo",
                "Ofertas de reenganche",
                "Excluir de campañas costosas",
                "Depuración del CRM"
            ]
        }.get(segment, [])


st.title("Segmentación de Clientes – Retail (RFM + K-Means)")

uploaded_file = st.file_uploader("Cargar dataset transaccional")

if uploaded_file:
    df = load_dataset(uploaded_file)

    customer_col = st.selectbox("Columna ID Cliente", df.columns)
    date_col = st.selectbox("Columna Fecha", df.columns)
    amount_col = st.selectbox("Columna Monto", df.columns)

    rfm = calculate_rfm(df, customer_col, date_col, amount_col)
    scaled = scale_features(rfm)

    model, labels = train_kmeans_fixed(scaled)
    rfm['Cluster'] = labels

    rfm = assign_rfm_segments(rfm)

    st.subheader("Distribución de Segmentos")
    st.bar_chart(rfm['Segmento'].value_counts())

    fig = px.scatter_3d(
        rfm,
        x='Recency',
        y='Frequency',
        z='Monetary',
        color='Segmento',
        title="Visualización 3D de Segmentación de Clientes"
    )
    st.plotly_chart(fig)

    # ----------------------------
    # RECOMENDACIONES
    # ----------------------------
    st.subheader("Recomendaciones de Marketing")

    selected_segment = st.selectbox(
        "Selecciona un segmento",
        ['VIP', 'Leales', 'En Riesgo', 'Perdidos']
    )

    recs = marketing_recommendations(selected_segment)

    for r in recs:
        st.markdown(f"✅ {r}")
""")

# Ahora, ejecuta el archivo app.py con streamlit
!streamlit run app.py
```

```
NgrokTunnel: "https://overcoolly-cycadaceous-colten.ngrok-free.dev" -> "http://localhost:8501"

Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.


  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.28.0.12:8501
  External URL: http://34.46.44.211:8501
```

Start coding or generate with AI.